Cairo University
Faculty of Computer and Artificial Intelligence

CS453: Software Testing and Quality Assurance
2025-2026

Assignment 1
Part 1: Java Math Library

Course Instructor
Dr. Manar ElKady

Team Members:

Name	ID
Ebaa Mostafa	20220001
Touka Mohamed	20220094
Yasmine Sherif	20220377
Muhammad Hassan	20221120

CS453: Software Testing and Quality Assurance 1
1. Introduction 4
2. Overview of the Tested Classes 4
2.1. StringUtil Class 4
2.2. Multiset_HashMapImpl Class 4
3. Testing Methodology 4
3.1. Techniques 4
3.2. Coverage Criteria 5
4. Test Design 5
4.1. StringUtil 5
Task I: Determine Characteristics 5
Identify Methods (Table A) 5
Design Partitioning (Table B) 6
Task II: Define Test Requirements 6
Test Requirements Using Base Choice Criteria (BCC) (Table C) 6
Boundary Analysis (Table D) 7
Task III: Test Cases (BCC+BVA) 8
Method 1: repeat() 8
Method 2: formatLeft() 8
Method 3: formatRight() 9
4.2. Multiset_HashMapImpl 10
Task I: Determine Characteristics 10
Identify Methods (Table A) 10
Design Partitioning (Table B) 12
Task II: Define Test Requirements 14
Test Requirements Using Base Choice Criteria (BCC) (Table C) 14
Boundary Analysis (Table D) 15
Task III: Test Cases (BCC+BVA) 17
Method 1: add(T entry) 17
Method 2: add(T entry, int mult) 18
Method 3: addAll(Multiset<T> other) 18
Method 4: addAll(Collection<T> values) 19

Method 5: addAll(T[] values) 19
Method 6: remove(Object key) 19
Method 7: removeAll(T key, int mult) 20
Method 8: removeAll(T key) 20
Method 9: intersect(Multiset<T> other) 21
Method 10: totalCount() 21
Method 11: toList() 22
Method 12: toString() 22
Method 13: equals(Object o) 22

1. Introduction

This report presents the design of test cases for selected classes from the *Java Math Library*. The purpose of testing is to verify that the implemented methods behave correctly for a wide range of inputs, including normal, edge, and invalid cases.

The test design follows a **black-box** testing approach, where test cases are derived from the method specifications without considering the internal implementation. This ensures that testing is based solely on expected behavior from the user's perspective.

---

2. Overview of the Tested Classes

2.1. StringUtil Class

- Utility class for string manipulation
- Methods tested:
  - `repeat ->` Concatenates string s n times
  - `formatLeft ->` Inserts a string s left-aligned into a mask, without truncation.
  - `formatRight ->` Inserts a string s right-aligned into a mask, without truncation.

All of the methods return a string (or null in case of repeat), and none of them throw exceptions.

2.2. Multiset_HashMapImpl Class

- Data structure based on HashMap
- Stores elements with multiplicity

---

3. Testing Methodology

3.1. Techniques

- **Input Space Partitioning (ISP):**
  The input domain of each method is divided into **partitions** such that all values within a partition are expected to behave similarly. Instead of testing all possible inputs, one representative value from each partition is selected, reducing the number of test cases while maintaining effectiveness.

- **Boundary Value Analysis (BVA):**
  Since errors are more likely to occur at the edges of input ranges, test cases are designed to target **boundary values**, such as minimum, maximum, and just-inside/outside limits. This helps detect faults that may not appear in general cases.

3.2. Coverage Criteria

Base Choice Coverage (BCC) is used, where a base test case with typical valid inputs is selected, and then one parameter is varied at a time. This ensures controlled and thorough exploration of the input space.

Justification: All Combinations of Choices (ACoC) would guarantee full coverage; however, it is costly and unnecessary. BCC is used to reduce the number of test cases while still ensuring that each partition is exercised.

By combining ISP, BVA, and BCC, the resulting test suite provides comprehensive coverage of both input conditions and expected functional behavior.

4. Test Design

4.1. StringUtil

Task I: Determine Characteristics

Identify Methods (Table A)

Method	Inputs	Returns	Values	Exceptions	Ch ID	Characteristics	Covered by
repeat	s: String, n: int	String or Null	null, empty string, repeated string	None	C1	Value of n with respect to 0	
					C2	Nullity of s	
					C3	Length of s	
formatLeft	s: String, mask: String	String	formatted string, empty string	None		Nullity of s	C2
					C4	Nullity of mask	
					C5	Length of s with respect to length of the mask	
formatRight	s: String, mask: String	String	formatted string, empty string	None		Nullity of s	C2
						Nullity of mask	C4

						Length of s with respect to length of mask	C5

Design Partitioning (Table B)

Ch ID	Characteristic	repeat()	formatLeft()	formateRight()	Partition
C1	Value of n with respect to 0	x			**B1:** n<0, **B2:** n=0, **B3:** n>0
C2	Nullity of s	x	x	x	**B1:** Null, **B2:** Not Null
C3	Length of s	x			**B1:** len(s) = 0, **B2:** len(s) > 0
C4	Nullity of mask		x	x	**B1:** Null, **B2**: Not Null
C5	Length of s with respect to length of mask		x	x	**B1:** len(s) < len(mask) **B2:** len(s) = len(mask) **B3:** len(s) > len(mask)

Task II: Define Test Requirements

Test Requirements Using Base Choice Criteria (BCC) (Table C)

Method	Characteristics	Test Requirements	Infeasible TRs	Revised TRs	# of TRs
repeat()	C1 C2 C3	**Base: B3, B2, B2** Vary: • B3, B2, B1 • B2, B2, B2 • B1, B2, B2	• B3, B1, B2 (s is null and has length)	• B3, B1, B1	5
formatLeft()	C2 C4 C5	**Base: B2, B2, B1** Vary: • B2, B2, B2 • B2, B2, B3 • B1, B2, B1	• B2, B1, B1 (mask is null and len(s) < len(mask))	• B2, B1, B3	5
formatRight()	C2 C4 C5	**Base: B2, B2, B1** Vary: • B2, B2, B2 • B2, B2, B3 • B1, B2, B1	• B2, B1, B1 (mask is null and len(s) < len(mask))	• B2, B1, B3	5

Boundary Analysis (Table D)

Method	Parameters	Boundary Values	Justification
repeat()	n	• n = 0 (boundary) • n = -1 (just below) • n = 1 (just above)	• n = 0 and n = -1: return null. • n = 1: the min value to return a string.
	s	• len(s) = 0 (“”) (boundary) • len(s) = 1 (just above) **No upper bound**	• len(s) = 0: the smallest valid length; it returns “” given any n value. • len(s) = 1: the min valid length for s.
formatLeft()	s	• len(s) = 0 (“”) (boundary) • len(s) = 1 (just above) **No upper bound**	• len(s) = 0: the smallest valid length, and returns the mask as it is. • len(s) = 1: the min valid length for s.
	mask	• len(mask) = 0 (“”) (boundary) • len(mask) = 1 (just above) **No upper bound**	• len(mask) = 0: the smallest valid length, and returns “s” as it is. • len(mask) = 1: the min valid length for mask.
	len(s) related to len(mask)	• len(s) = len(mask) (boundary) • len(s) = len(mask) + 1 (just below) • len(s) = len(mask) - 1 (just above)	• len(s) = len(mask) and len(s) = mask + 1: both return the string s as it is • len(s) = len(mask) + 1: returns a combination of the string followed by the last character in the mask.
formatRight()	s	• len(s) = 0 (“”) (boundary) • len(s) = 1 (just above) **No upper bound**	• len(s) = 0: the smallest valid length, and returns the mask as it is. • len(s) = 1: the min valid length for s.
	mask	• len(mask) = 0 (“”) (boundary) • len(mask) = 1 (just above) **No upper bound**	• len(mask) = 0: the smallest valid length, and returns “s” as it is. • len(mask) = 1: the min valid length for mask.
	len(s) related to len(mask)	• len(s) = len(mask) (boundary) • len(s) = len(mask) + 1 (just below) • len(s) = len(mask) - 1 (just above)	• len(s) = len(mask) and len(s) = mask + 1: both return the string s as it is • len(s) = len(mask) + 1: returns a combination of the string followed by the last character in the mask.

Task III: Test Cases (BCC+BVA)

Method 1: repeat()

TC	s	n	Expected	Covered By
TC1	“abc”	2	“abcabc”	TR1 (base test)
TC2	“abc”	1	“abc”	TR1, (B: n = 1)
TC3	“”	2	“”	TR2, (B: s = “”)
TC4	“abc”	0	null	TR3, (B: n = 0)
TC5	“abc”	-1	null	TR4, (B: n = -1)
TC6	null	2	null	TR5
TC7	“a”	2	“aa”	(B: len(s) = 1)

Method 2: formatLeft()

TC	s	mask	Expected	Covered By
TC1	“abc”	“123456”	“abc456”	TR1 (base test)
TC2	“a”	“1”	“a”	TR2, (B: len(s) = 1, len(mask) = 1, len(s) = len(mask))
TC3	“abcd”	“”	“abcd”	TR3, (B:mask = “”)
TC4	null	“12345”	“12345”	TR4

TC5	“abcd”	null	“abcd”	TR5
TC6	“”	“1”	“1”	(B: s = “”, len(s) = len(mask) -1)
TC7	“abcd”	“123”	“abcd”	(B: len(s) = len(mask) + 1)

Method 3: formatRight()

TC	s	mask	Expected	Covered By
TC1	“abc”	“123456”	“123abc”	TR1 (base test)
TC2	“a”	“1”	“a”	TR2, (B: len(s) = 1, len(mask) = 1, len(s) = len(mask))
TC3	“abcd”	“”	“abcd”	TR3, (B:mask = “”)
TC4	null	“12345”	“12345”	TR4
TC5	“abcd”	null	“abcd”	TR5
TC6	“”	“1”	“1”	(B: s = “”, len(s) = len(mask) -1)
TC7	“abcd”	“123”	“abcd”	(B: len(s) = len(mask) + 1)

4.2. Multiset_HashMapImpl

Task I: Determine Characteristics

Identify Methods (Table A)

Method	Inputs	Returns	Exceptions	Ch ID	Characteristics	Covered by
add(T entry)	entry: T (generic), state	int	None	C1	Entry existence in the multiset	
add(T entry, int mult)	entry: T, mult: int, state	int	None		Entry existence in the multiset	C1
				C2	Mult value related to 0	
addAll(Multiset<T> other)	other: Multiset<T>, state	void	None	C3	Nullity of other (multiset)	
				C4	Size of other (multiset)	
addAll(Collection <T> values)	values: T[], state	void	None	C5	Nulity of values (collection)	
				C6	Size of values (collection)	
addAll(T[] values)	key: Object, state	void	None	C7	Nullity of values(array)	
				C8	Size of values (array)	
remove(Object key)	key: Object, state	Integer, null	None		Key existence in the multiset	C1
				C9	Multiplicity of element if existing (>1)	
remove(T key, int mult)	key: T, mult: int, state	int	None		Key existence in the multiset	C1
				C10	Mult relative to old multiplicity	
removeAll(T key)	key: T,	int	None		Key existence in the multiset	C1

	state					
intersect(Multiset <T> other)	other: Multiset<T>, state	MultiSet<T>	None		Nullity of other (multiset)	C3
				C11	Multiplicity of elements in other and in multiset	
totalCount()	state	int	None	C12	Size of multiset	
toList()	state	List<T>	None		Size of multiset	C12
toString()	state	String	None		Size of multiset	C12
equal(Object o)	o: Object, state	boolean	None	C13	Nullity of o (Object)	
				C14	Type of o (instance of)	
				C15	Size of o relative to multiset	
					Multiplicity of elements in o and in multiset	C11

Design Partitioning (Table B)

Ch ID	dd(T)	add(T, int)	addAll (Multiset)	addAll(Collection)	addAll(T[])	rem(Obj)	rem(T, int)	remAll (T)	intersect()	totalCnt()	toLst()	toStr()	eq()	Partitions
C1	x	x				x	x	x						**B1:** entry exists in multiset **B2:** entry doesn't exist in multiset
C2		x												**B1:** mult < 0 **B2:** mult = 0 **B3:** mult > 0
C3			x						x					**B1:** other is null **B2:** other is non null
C4			x											**B1:** size other = 0 **B2:** size other > 0
C5				x										**B1:** values is null **B2:** values is non null
C6				x										**B1:** size values = 0 **B2:** size values > 0
C7					x									**B1:** values is null **B2:** values is non null
C8					x									**B1:** size values = 0 **B2:** size values > 0

C9						x								**B1:** multiplicity = 1 **B2:** multiplicity > 1
C10							x							**B1:** mult < old multiplicity **B2:** mult = old multiplicity **B3:** mult > old multiplicity
C11									x				x	**B1:** multiplicity < other multiplicity **B2:** multiplicity = other multiplicity **B3:** multiplicity > other multiplicity
C12										x	x	x		**B1:** multiset size = 0 **B2:** multiset size > 0
C13													x	**B1:** o is null **B2:** o is non null
C14													x	**B1:** o is an instance of multiset **B2:** o isn't an instance of multiset
C15													x	**B1:** size o < size multiset **B2:** size o = size multiset **B3:** size o > size multiset

Task II: Define Test Requirements

Test Requirements Using Base Choice Criteria (BCC) (Table C)

Method	Characteristics	Test Requirements	Infeasible TRs	Revised TRs	# of TRs
add(T entry)	C1	**Base: B1** Vary: • B2			2
add(T entry, int mult)	C1 C2	**Base: B1 B3** Vary: • B1 B1 • B1 B2 • B2 B3			4
addAll(Multiset <T> other)	C3 C4	**Base: B2 B2** Vary: • B2 B1	• B1 B2 (other is null and has length)	• B1 B1	3
addAll(Collection<T> values)	C5 C6	**Base: B2 B2** Vary: • B2 B1	• B1 B2 (values is null and has length)	• B1 B1	3
addAll(T[] values)	C7 C8	**Base: B2 B2** Vary: • B2 B1	• B1 B2 (values is null and has length)	• B1 B1	3
remove(Object key)	C1 C9	**Base: B1 B2** Vary: • B1 B1	• B2 B2	none	2
remove (T key, int mult)	C1 C10	**Base: B1 B1** Vary: • B1 B2 • B1 B3 • B2 B1			4
removeAll(T key)	C1	**Base: B1** Vary: • B2			2

intersect(Multiset<T> other)	C3 C11	**Base: B2 B2** Vary: • B2 B1 • B2 B3	• B1 B2	• B1 B3	4
totalCount()	C12	**Base: B2** Vary: • B1			2
toList()	C12	**Base: B1** Vary: • B2			2
toString()	C12	**Base: B1** Vary: • B2			2
equal(Object o)	C11 C13 C14 C15	**Base: B2 B2 B1 B2** Vary: • B2 B2 B1 B3 • B2 B2 B2 B2 • B1 B2 B1 B2 • B3 B2 B1 B2	• B2 B2 B1 B1 • B2 B1 B1 B2 (nullity of caller)	• B3 B1 B1 B2	6

Boundary Analysis (Table D)

Method	Parameteres	Boundary Values	Justification
add(T entry)	entry	• Entry multiplicity in set = 1 (boundary) • Entry multiplicity in set = 2 (just above)	• Adding the element for the first time • incrementing multiplicity
	size(multiset)	• Multiset is empty (boundary)	• Test the behaviour of the first element by adding
add(T entry, int mult)	entry	• Entry multiplicity = 1 (boundary) • Entry multiplicity = 2 (just above)	• Adding the element for the first time • Incrementing multiplicity
	mult	• Mult = -1 (just below) • Mult = 0 (boundary) • Mult = 1 (just above)	• 0 and -1: not changing the set • 1: adding the min amount of element successfully behaviour

addAll(Multiset<T> other)	other	• size(other) = 0 (boundary) • size(other) = 1 (just above)	• Adding no elements • Adding the min amount of elements
addAll(Collection<T> values)	values	• size(values) = 0 (boundary) • size(values) = 1 (just above)	• Adding no elements • Adding the min amount of elements
addAll(T[] values)	values	• size(values) = 0 (boundary) • size(values) = 1 (just above)	• Adding no elements • Adding the min amount of elements
remove(Object key)	key	• Multiplicity of key is null (just below) • Multiplicity of key = 1 (boundary) • Multiplicity of key = 2 (just above)	• Mult is null: not deleting any element, returns null • Mult = 1: removing key from set • Mut = 2: decrementing key multiplicity by 1
remove (T key, int mult)	key	• Multiplicity of key is null	• Not deleting any key
	mult	• Mult = -1 (just below) • Mult = 0 (boundary) • Mult = 1 (just above)	• Mult = -1 and mult = 0: not deleting any elements, return 0 • Mult = 1: min value that deletes an element
	Key multiplicity related to mult	• Multiplicity of key = mult - 1 (just below) • Multiplicity of key = mult (boundary) • Multiplicity of key = mult + 1 (just above)	• Multiplicity = mult or mult -1: removing the key from set • Multiplicity = mult + 1: decrementing key multiplicity
removeAll(T key)	key	• Multiplicity of key is null (just below) • Multiplicity of key = 1 (boundary) • Multiplicity of key = 2 (just above)	• Mult is null: not deleting any element • Mult = 1: removing key from set • Mut = 2: decrementing key multiplicity by 1
intersect(Multiset<T> other)	other	• size(other) = 0 (boundary) • size(other) = 1 (just above)	• return empty multiset • min size of possible non-zero intersections
	Key multiplicity related to	• Multiplicity of key = Multiplicity of multiset - 1 (just below) • Multiplicity of key = Multiplicity	• Multiplicity of key = multiset - 1 and m + 1: taking the min value • Multiplicity of key = multiset: fully

	multiset	of multiset (boundary) • Multiplicity of key = Multiplicity of multiset + 1 (just above)	intersect
totalCount()	state	• size(multiset) = 0 (boundary) • size(multiset) = 1 (just above)	• If set is empty, return 0 • 1: returns the multiplicity of the min (1) element existing in set
toList()	state	• size(multiset) = 0 (boundary) • size(multiset) = 1 (just above)	• If set is empty, return 0 • 1: returns a list of min size (1) in set
toString()	state	• size(multiset) = 0 (boundary) • size(multiset) = 1 (just above)	• If set is empty, return 0 • 1: returns list of min size (1) in set
equal(Object o)	o	• size(o) = 0 (boundary) • size(o) = 1 (just above)	
	o multiplicity related to multiset	• Multiplicity of o = Multiplicity of multiset - 1 (just below) • Multiplicity of o = Multiplicity of multiset (boundary) • Multiplicity of o = Multiplicity of multiset + 1 (just above)	• Multiplicity of key = Multiplicity of set -1 or + 1: returning false on any element not existing with the same multiplicity in both • Multiplicity of key = Multiplicity of set: happy scenario where elements have the same multiplicity in both
	o size-related to multiset	• size(o) = size(multiset) - 1 (just below) • size(o) = size(multiset) (boundary) • size(o) = size(multiset) + 1(just above)	• size(o) = size(set) -1 or + 1 test returning false • size(o) = size(set): happy scenario where both sets are of the same size

Task III: Test Cases (BCC+BVA)

Method 1: add(T entry)

TC	Initial multiset	entry	Expected	Covered By
TC1	{“abc”, “efg”}	“abc”	{“abc” = 2, “efg”}	TR1, (B: entry multi = 1)
TC2	{“efg”}	“abc”	{“abc”, “efg”}	TR2
TC3	{“abc” = 2, “efg”}	“abc”	{“abc” = 3, “efg”}	(B: entry multi = 2)

TC4	{}	“abc”	{“abc”}	(B: multiset is empty)

Method 2: add(T entry, int mult)

TC	Initial multiset	entry	mult	Expected	Covered By
TC1	{“abc” = 2, “efg”}	“abc”	2	{“abc” = 4, “efg”}	TR1, (B: multiplicity in set = 2)
TC2	{“abc”, “efg”}	“abc”	-1	{“abc”, “efg”}	TR2, (B: mult = -1), (B: multiplicity in set = 1)
TC3	{“abc” = 2, “efg”}	“abc”	0	{“abc” = 2, “efg”}	TR3, (B: mult = 0)
TC4	{“efg”}	“abc”	1	{“abc”, “efg”}	TR4, (B: mult = 1)

Method 3: addAll(Multiset<T> other)

TC	Initial multiset	other	Expected	Covered By
TC1	{“abc” = 2, “efg”}	{“abc”, “hij”}	{“abc” = 3, “efg”, “hij”}	TR1
TC2	{“abc”, “efg”}	{}	{“abc”, “efg”}	TR2, (B: size(other) = 0)
TC3	{“abc”, “efg”}	null	{“abc”, “efg”}	TR3
TC4	{}	{“abc”}	{“abc”}	(B: size(other) = 1)

Method 4: addAll(Collection<T> values)

TC	Initial multiset	values	Expected	Covered By
TC1	{“abc” = 2, “efg”}	{“abc”, “hij”}	{“abc” = 3, “efg”, “hij”}	TR1
TC2	{“abc”, “efg”}	{}	{“abc”, “efg”}	TR2, (B: size(other) = 0)
TC3	{“abc”, “efg”}	null	{“abc”, “efg”}	TR3
TC4	{}	{“abc”}	{“abc”}	(B: size(other) = 1)

Method 5: addAll(T[] values)

TC	Initial multiset	other	Expected	Covered By
TC1	{“abc” = 2, “efg”}	[“abc”, “hij”]	{“abc” = 3, “efg”, “hij”}	TR1
TC2	{“abc”, “efg”}	[]	{“abc”, “efg”}	TR2, (B: size(other) = 0)
TC3	{“abc”, “efg”}	null	{“abc”, “efg”}	TR3
TC4	{}	[“abc”]	{“abc”}	(B: size(other) = 1)

Method 6: remove(Object key)

TC	Initial multiset	key	Expected Return	Expected	Covered By
TC1	{“abc” = 2, “efg”}	“abc”	2	{“abc”, “efg”}	TR1, (B: Multiplicity of key = 2)

TC2	{“abc”, “efg”}	“abc”	1	{“efg”}	TR2, (B: Multiplicity of key = 1)
TC3	{“efg”}	“abc”	null	{“efg”}	(B: Multiplicity of key is null)

Method 7: removeAll(T key, int mult)

TC	Initial multiset	key	mult	Expected Return	Expected	Covered By
TC1	{“abc” = 3, “efg”}	“abc”	2	3	{“abc”, “efg”}	TR1, (B: multiplicity of key = mult + 1)
TC2	{“abc”, “efg”}	“abc”	1	1	{“efg”}	TR2, (B: multiplicity of key = 1), (B: multiplicity of key = mult)
TC3	{“abc” = 2, “efg”}	“abc”	3	2	{“efg”}	TR3, (B: multiplicity of key = mult - 1)
TC4	{“efg”}	“abc”	2	0	{“efg”}	TR4, (B: multiplicity of key is null)
TC5	{“abc” = 2, “efg”}	“abc”	-1	2	{“abc” = 2, “efg”}	(B: mult = -1)
TC6	{“abc” = 2, “efg”}	“abc”	0	2	{“abc” = 2, “efg”}	(B: mult = 0)

Method 8: removeAll(T key)

TC	Initial multiset	key	Expected Return	Expected	Covered By
TC1	{“abc” = 2, “efg”}	“abc”	2	{“efg”}	TR1, (B: Multiplicity of key = 2)
TC2	{“efg”}	“abc”	0	{“efg”}	TR2, (B: Multiplicity of key is null)
TC3	{“abc”, “efg”}	“abc”	1	{“efg”}	(B: Multiplicity of key = 1)

Method 9: intersect(Multiset<T> other)

TC	Initial multiset	other	Expected	Covered By
TC1	{“abc” = 2, “efg”}	{“abc” = 2}	{“abc” = 2}	TR1, (B: Multiplicity of other = Multiplicity of multiset)
TC2	{“abc”, “efg”}	{“abc” = 2, “efg”}	{“abc”, “efg”}	TR2, (B: Multiplicity of other = Multiplicity of multiset + 1)
TC3	{“abc” = 2, “efg”}	{“abc”}	{“abc”}	TR3, (B: size(other) = 1), (B:Multiplicity of other = Multiplicity of multiset - 1)
TC4	{“abc”}	null	{}	TR4
TC5	{“abc”, “efg”}	{}	{}	(B: size(other) = 0)

Method 10: totalCount()

TC	Initial multiset	Expected	Covered By
TC1	{“abc” = 2, “efg”}	3	TR1
TC2	{}	0	TR2, (B: size = 0)
TC3	{“abc”}	1	(B: size = 1)

Method 11: toList()

TC	Initial multiset	Expected	Covered By
TC1	{“abc” = 2, “efg”}	[“abc”, “abc”, “efg”]	TR1
TC2	{}	[]	TR2, (B: size = 0)
TC3	{“abc”}	[“abc”]	(B: size = 1)

Method 12: toString()

TC	Initial multiset	Expected	Covered By
TC1	{“abc” = 2, “efg”}	{abc^2, efg}	TR1
TC2	{}	{}	TR2, (B: size = 0)
TC3	{“abc”}	{abc}	(B: size = 1)

Method 13: equals(Object o)

TC	Initial multiset	o	Expected	Covered By
TC1	{“abc” = 2, “efg”}	{“abc” = 2, “efg”}	true	TR1, (B: Multiplicity of o = Multiplicity of multiset)
TC2	{“abc”}	{“abc”, “efg”}	false	TR2, (B: size(o) = size(multiset) + 1))
TC3	{“abc”, “efg”}	[“abc”, “efg”]	false	TR3
TC4	{“abc”}	null	false	TR4, (B: size(o) = 0)

TC5	{“abc” = 2, “efg”}	{“abc”, “efg”}	false	TR5, (B: Multiplicity of o = Multiplicity of multiset - 1)
TC6	{“abc”, “efg”}	{“abc” = 2, “efg”}	false	TR6, (B: Multiplicity of o = Multiplicity of multiset + 1), (B: size(o) = size(multiset))
TC7	{“abc”, “efg”}	{“abc”}	false	(B: size(o) = 1), (B: size(o) = size(multiset) - 1)